

Scheduling Metrics and Workload Characterization

CPU Scheduling – Unit 2

Prashant Jadav

Fundamentals of Operating System Design (FOSD)

Learning Objectives

By the end of this topic, students should be able to:

- Explain why CPU scheduling requires quantitative evaluation.
- Define turnaround time, waiting time, response time and throughput.
- Calculate CPU utilization and scheduling efficiency.
- Distinguish between preemptive and non-preemptive workload behavior.
- Characterize workloads using CPU burst, I/O burst, arrival rate and concurrency.
- Analyze how workload characteristics influence scheduler design.
- Select appropriate scheduling metrics for different systems.

Why Do We Need Scheduling Metrics?

The CPU scheduler decides **which runnable execution entity gets CPU time**.

But how do we determine whether a scheduling policy is effective?

Scheduling metrics

Quantitative measures used to evaluate the behavior and performance of a CPU scheduling policy.

Typical questions

- How quickly does a process complete?
- How long does it wait?
- How quickly does it start responding?
- How much CPU time is actually doing useful work?
- How many jobs can the system complete?

Key idea

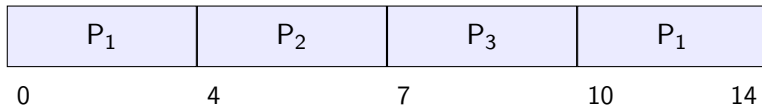
Different systems optimize different metrics. A server, interactive desktop and real-time controller do not necessarily have the same scheduling objectives.

CPU Scheduling: Basic Timeline

Consider three processes:

P_1, P_2, P_3

A CPU schedule can be represented using a **Gantt chart**.



From such a timeline we can calculate:

Turnaround, Waiting, Response, Throughput

Process Timing Parameters

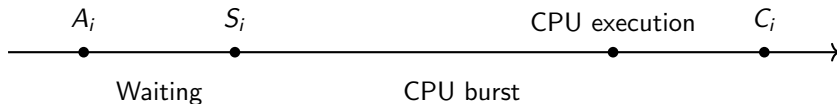
For each process P_i , define:

A_i = Arrival Time

B_i = CPU Burst Time

S_i = First CPU Start Time

C_i = Completion Time



These four times form the basis of several scheduling metrics.

1. Turnaround Time

Definition

Turnaround time is the total time taken by a process from its arrival/submission until its completion.

$$TAT_i = C_i - A_i$$

It includes:

Turnaround Time = Waiting + CPU Execution + I/O/Blocked Time

Example

If:

$$A_i = 2, \quad C_i = 14$$

then:

$$TAT_i = 14 - 2 = 12$$

Interpretation

Turnaround Time: Real-World Analogy

Example: Restaurant order

A customer places an order at 12:00.

The order is ready at 12:25.

Turnaround Time = 25 minutes

It includes:

- Waiting before preparation starts.
- Actual preparation.
- Any delays during the process.

OS analogy

Customer arrival → process arrival

Food preparation → CPU execution

Order completion → process completion

This analogy helps distinguish turnaround time from CPU burst time.

2. Waiting Time

Definition

Waiting time is the total time a process spends waiting in the ready queue for CPU allocation.

For a simple CPU-burst model:

$$WT_i = TAT_i - B_i$$

Therefore:

$$WT_i = (C_i - A_i) - B_i$$

Important distinction

Waiting Time $\neq$ Total Time Not Running

In the standard CPU scheduling metric, waiting time refers specifically to time spent in the **ready queue**.

Waiting Time: Example

Suppose:

$$A = 0, \quad B = 5, \quad C = 12$$

Then:

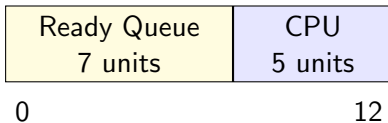
$$TAT = C - A = 12$$

and:

$$WT = TAT - B$$

$$WT = 12 - 5 = \boxed{7}$$

Timeline:



The process spends 7 units waiting and 5 units executing.

3. Response Time

Definition

Response time is the time from process arrival until the process receives CPU service for the first time.

$$RT_i = S_i - A_i$$

Response time is especially important for:

- Interactive applications
- Web interfaces
- Terminal systems
- Mobile applications
- Real-time user interaction

Important

Response time is **not** the same as turnaround time. A process may start quickly but finish much later.

Response Time: Interactive Example

Imagine typing a command into a terminal.

Keyboard input → OS receives request → CPU starts processing

If the CPU starts processing after:

50 ms

then:

$RT = 50\ ms$

Even if the command requires:

2 seconds

to finish, the user may perceive the system as responsive because it started quickly.

Design implication

Interactive systems often prioritize low response time rather than only minimizing average turnaround time.

4. Throughput

Definition

Throughput is the amount of completed work during a unit of time.

For completed processes:

$$\textit{Throughput} = \frac{\text{Number of completed processes}}{\text{Observation time}}$$

Example

A server completes:

600 requests in 60 seconds

Then:

$$\textit{Throughput} = \frac{600}{60} = 10 \text{ requests/sec}$$

Interpretation

Higher throughput means the system completes more work per unit time, but it does not necessarily mean every individual process experiences low response or turnaround time.

5. CPU Utilization

Definition

CPU utilization is the fraction of time the CPU is busy doing useful or system work during an observation interval.

$$CPU\ Utilization = \frac{CPU\ Busy\ Time}{Total\ Observation\ Time} \times 100$$

Example

CPU busy:

950 ms

during:

1000 ms

Therefore:

$$CPU\ Utilization = \frac{950}{1000} \times 100 = \boxed{95\%}$$

6. Fairness

Definition

Fairness describes how evenly CPU service is distributed among competing runnable entities according to the system's scheduling policy.

A scheduler may consider:

- Priority
- CPU usage
- Waiting time
- User/service class
- Deadlines
- Resource entitlements

Example

If one low-priority process waits indefinitely while higher-priority processes repeatedly execute, starvation may occur.

7. Starvation and Scheduling Metrics

Starvation

A process experiences starvation when it waits for an unreasonably long time because scheduling decisions repeatedly favor other processes.

Possible indicators

- Very large waiting time.
- Very large response time.
- Very low CPU share.
- Large variation in service received.

Possible solution

Aging gradually increases the effective priority of a waiting process so that it eventually receives service.

8. Normalized / Relative Turnaround Time

Sometimes turnaround time is compared with the actual CPU service required.

Normalized turnaround time

$$NTAT_i = \frac{TAT_i}{B_i}$$

Example

If:

$$TAT = 20, \quad B = 5$$

then:

$$NTAT = \frac{20}{5} = 4$$

Interpretation: The process spent four times its CPU burst duration in the system.

Use

This metric helps compare the experience of jobs with different CPU burst lengths.

9. Scheduling Efficiency

A simple direct-overhead view is:

$$\text{Efficiency} = \frac{\text{Useful CPU Work}}{\text{Total CPU Time}} \times 100$$

Suppose:

$$T_{\text{useful}} = 920 \text{ ms}$$

and:

$$T_{\text{overhead}} = 80 \text{ ms}$$

Then:

$$T_{\text{total}} = 1000 \text{ ms}$$

$$\text{Efficiency} = \frac{920}{1000} \times 100 = \boxed{92\%}$$

Overhead can include:

- Context switching
- Scheduler execution

Metric Summary

Metric	Formula	Primary meaning
Turnaround	$C - A$	Total time in system
Waiting	$TAT - B$	Time waiting in ready queue
Response	$S - A$	Time until first CPU service
Throughput	Completed / Time	Work completed per unit time
CPU Utilization	Busy / Total	CPU activity level
Normalized TAT	TAT / B	Relative completion delay
Fairness	Policy-dependent	Distribution of CPU service
Efficiency	Useful / Total	Fraction spent on useful work

Which Metric Should We Optimize?

There is no single universally optimal metric.

Interactive desktop

- Response time
- Fairness
- Responsiveness

Batch system

- Throughput
- Turnaround time

Server

- Throughput
- Response time
- CPU utilization

Real-time system

- Deadline satisfaction
- Predictability
- Worst-case response

Design principle

The appropriate metric depends on the system's **requirements and workload**.

What Is Workload Characterization?

Definition

Workload characterization describes the properties of the computational work presented to the operating system.

A workload can be described using:

- Number of processes/threads
- Arrival pattern
- CPU burst lengths
- I/O burst behavior
- Degree of concurrency
- Priority distribution
- Interactive vs. batch behavior
- CPU-bound vs. I/O-bound behavior

Why characterize workload?

A scheduling algorithm that works well for one workload may perform poorly for another.

CPU-Bound vs. I/O-Bound Workloads

CPU-Bound

Most execution time is spent performing computation.

Examples:

- Scientific simulation
- Video encoding
- Large mathematical computation
- Machine-learning computation

I/O-Bound

Processes frequently wait for I/O.

Examples:

- Database access
- File processing
- Network applications
- Interactive applications

CPU-bound: Long CPU bursts

I/O-bound: Short CPU bursts + frequent I/O

CPU-Bound Workload: Example

Consider a video-encoding application.



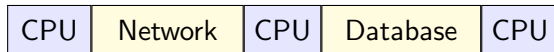
Most time is spent on CPU computation.

Scheduling implication

Keeping the CPU busy and maintaining high throughput may be particularly important.

I/O-Bound Workload: Example

Consider a web server processing requests.



The process frequently blocks for external operations.

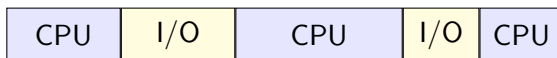
Scheduling implication

A scheduler can improve CPU utilization by running other ready tasks while one task waits for I/O.

CPU Burst and I/O Burst Cycle

Many processes alternate between CPU computation and I/O.

CPU Burst $\rightarrow$ I/O Wait $\rightarrow$ CPU Burst $\rightarrow$ I/O Wait $\rightarrow \dots$



Scheduling relevance

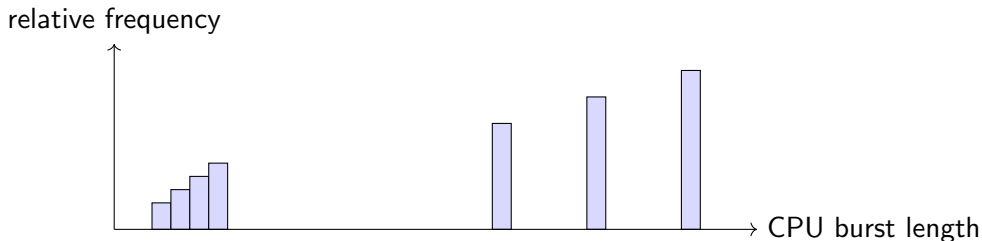
CPU burst distribution is one of the most important workload characteristics when analyzing CPU scheduling algorithms.

CPU Burst Distribution

Suppose a workload contains:

$$B = \{2, 3, 4, 5, 20, 25, 30\}$$

Most jobs have short CPU bursts, while a few have long bursts.
This creates a **mixed workload**.



Scheduling implication

Algorithms such as SJF/SRTF can exploit differences in burst lengths, but estimating future CPU bursts is itself a challenge.

Arrival Rate

Definition

Arrival rate is the average rate at which processes, threads or requests enter the system.

$$\lambda = \frac{\text{Number of arrivals}}{\text{Observation time}}$$

Example

If:

1200 requests arrive in 60 seconds

then:

$$\lambda = \frac{1200}{60} = 20 \text{ requests/sec}$$

Scheduling implication

Higher arrival rates can increase ready-queue length, waiting time and scheduling pressure.

Concurrency Level

Definition

Concurrency refers to the number of tasks that are active, runnable or in progress during a given period.

A workload can have:

Low concurrency or High concurrency

Example

- Desktop with a few active applications: moderate concurrency.
- Cloud server handling thousands of connections: high concurrency.

Important

High concurrency does not necessarily mean high CPU utilization. Many tasks may be blocked on I/O.

Interactive vs. Batch Workloads

Interactive

Examples:

- Terminal
- Web browser
- GUI applications
- Text editor

Important metrics:

- Response time
- Fairness
- Predictability

Batch

Examples:

- Payroll processing
- Large data conversion
- Scientific jobs
- Offline report generation

Important metrics:

- Throughput
- Turnaround time
- CPU utilization

Burstiness of Workload

Workload arrivals are not always evenly distributed.

Smooth workload:



Bursty workload:



Example

An e-commerce website may experience a large increase in requests during a sale or promotional event.

Scheduling implication

- Ready queues can suddenly grow.
- Response time can increase.
- CPU and I/O resources can become bottlenecks.

Priority Distribution

Processes may have different priorities.

For example:

P_1 : *High* P_2 : *Medium* P_3 : *Low*

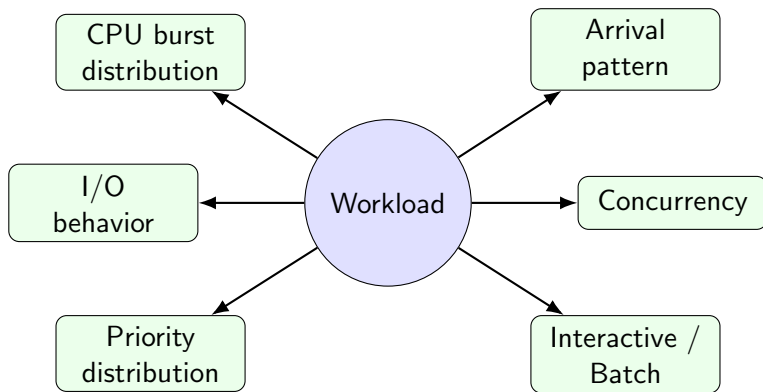
Priority can represent:

- System importance
- User preference
- Service-level requirement
- Real-time urgency
- Resource policy

Trade-off

Aggressive priority scheduling can reduce response time for high-priority tasks but may increase waiting time or starvation risk for low-priority tasks.

Workload Characterization Dimensions



Workload Characterization and Scheduling Algorithms

Workload property	Possible concern	Scheduling consideration
Many short jobs	Long jobs may delay short jobs	SJF/SRTF-type strategies
Interactive tasks	User-visible delay	Short response time
CPU-bound tasks	Long CPU occupancy	Fairness and throughput
I/O-bound tasks	Frequent blocking	Keep CPU busy with other tasks
High arrival rate	Ready queue grows	Queue management and scalability
Bursty arrivals	Temporary overload	Responsiveness and queue control
Priority classes	Unequal importance	Priority scheduling / aging
Deadlines	Late completion unacceptable	Real-time scheduling

End-to-End Example: Web Server

Consider a web server receiving requests.

Workload characteristics

- High request arrival rate.
- Many concurrent requests.
- Short CPU bursts.
- Frequent network/database I/O.
- Some requests may have different priorities.

Important metrics

Response Time + Throughput + CPU Utilization

Question

If the average response time suddenly increases while CPU utilization remains low, what workload or resource behavior would you investigate?

End-to-End Example: Video Encoding

Consider an offline video encoding server.

Workload characteristics

- Long CPU bursts.
- High computational intensity.
- Relatively low interaction with users.
- Multiple independent encoding jobs.

Important metrics

Throughput + Turnaround Time + CPU Utilization

Observation

A scheduler that slightly increases response time but substantially improves throughput may be acceptable for this workload.

End-to-End Example: Interactive Desktop

Consider a student using:

- Web browser
- Text editor
- Terminal
- Video player

The workload contains:

- Short CPU bursts.
- Frequent I/O waits.
- User-generated bursts of activity.
- Many concurrent background tasks.

Important metrics:

Response Time + Fairness + Responsiveness

Design implication

The system should feel responsive even when background processes are performing long computations.

Numerical Example: Complete Metric Calculation

Consider three processes:

Process	Arrival A	Burst B	Completion C
P_1	0	5	8
P_2	1	3	11
P_3	2	2	13

Calculate:

- 1 Turnaround time
- 2 Waiting time
- 3 Average turnaround time
- 4 Average waiting time

Formulas

$$TAT = C - A \quad WT = TAT - B$$

Numerical Example: Solution

For P_1 :

$$TAT_1 = 8 - 0 = 8$$

$$WT_1 = 8 - 5 = 3$$

For P_2 :

$$TAT_2 = 11 - 1 = 10$$

$$WT_2 = 10 - 3 = 7$$

For P_3 :

$$TAT_3 = 13 - 2 = 11$$

$$WT_3 = 11 - 2 = 9$$

Therefore:

$$\text{Average } TAT = \frac{8 + 10 + 11}{3} = 9.67$$

$$\text{Average } WT = \frac{3 + 7 + 9}{3} = 6.33$$

A Critical Observation About Metrics

Consider two scheduling algorithms:

Metric	Algorithm A	Algorithm B
Average waiting time	4	6
Average response time	2	3
Throughput	10 jobs/s	12 jobs/s
CPU utilization	90%	95%

Question

Which algorithm is better?

Answer: The answer depends on the system's requirements.

Lesson

Scheduling evaluation is a **multi-objective problem**. A single metric cannot completely describe scheduler quality.

Metric Selection by System Type

System	Primary metrics	Secondary metrics	Workload
Batch	Throughput, TAT	CPU utilization	Long jobs
Interactive	Response time	Fairness, TAT	Short/bursty jobs
Server	Throughput, response	Utilization	Concurrent requests
Real-time	Deadline satisfaction	Worst-case latency	Time-critical
Mobile	Response, energy	Fairness	Mixed workload

Common Mistakes in Scheduling Analysis

- ❶ Confusing waiting time with turnaround time.
- ❷ Treating response time as completion time.
- ❸ Forgetting process arrival times.
- ❹ Ignoring CPU idle periods.
- ❺ Assuming high CPU utilization means good performance.
- ❻ Comparing algorithms using only one metric.
- ❼ Ignoring workload characteristics.
- ❽ Assuming the same scheduler is optimal for every workload.

Exam tip

Always draw the Gantt chart first, then calculate the required metric systematically.

A Systematic Method for Scheduling Problems

- 1 List all processes and their arrival times.
- 2 Record CPU burst lengths.
- 3 Identify whether the algorithm is preemptive.
- 4 Construct the Gantt chart.
- 5 Determine first CPU start time S_i .
- 6 Determine completion time C_i .
- 7 Calculate:

$$TAT_i = C_i - A_i$$

$$WT_i = TAT_i - B_i$$

$$RT_i = S_i - A_i$$

- 8 Calculate averages if required.
- 9 Interpret the result using workload characteristics.

Higher-Order Thinking Questions

- 1 A scheduler reduces average waiting time but increases average response time. Explain how this can happen.
- 2 A system has 95% CPU utilization but poor user responsiveness. Identify possible reasons.
- 3 Two workloads have the same average CPU burst length, but one has highly variable burst lengths. Explain why the scheduling behavior may differ.
- 4 A server experiences a sudden burst of thousands of requests. Which workload characteristics have changed, and which scheduling metrics are likely to be affected?

Analytical Question

A university server has the following workload:

- 70% short interactive requests.
- 20% medium CPU-bound jobs.
- 10% long batch jobs.
- Bursty arrivals during examination periods.

Task

Characterize the workload and identify:

- 1 Three important scheduling metrics.
- 2 Two workload characteristics that may cause scheduling difficulty.
- 3 One possible trade-off the scheduler must manage.

Justify each answer using the workload properties.

Summary: Scheduling Metrics

Metric	Key question answered
Turnaround time	How long did the process take from arrival to completion?
Waiting time	How long did it wait in the ready queue?
Response time	How quickly did it first receive CPU service?
Throughput	How much work was completed per unit time?
CPU utilization	How busy was the CPU?
Fairness	How was CPU service distributed?
Efficiency	How much time was spent on useful work?

Summary: Workload Characterization

A workload can be characterized by:

Arrival + CPU bursts + I/O behavior + Concurrency + Priority + Interaction

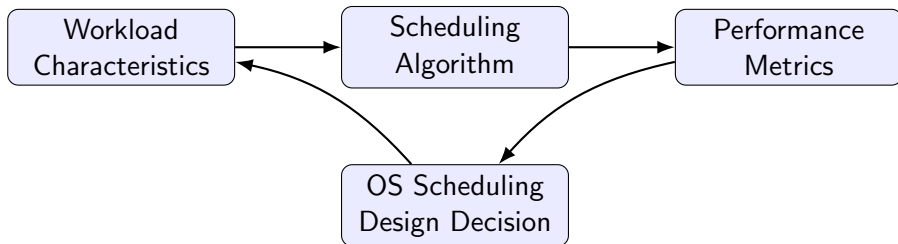
Key takeaway

The effectiveness of a scheduling algorithm depends strongly on the characteristics of the workload it serves.

Remember

Measure the right metric for the right workload.

Final Conceptual View



Workload → Scheduling → Measurement → Evaluation

A good scheduling analysis connects the workload, algorithm and measured outcome.